

ML ACADEMY

MLOps & Production AI Interview Handbook

Detailed Questions, Explanations & Worked Examples for Top Tech
Interviews

Pipelines • Experiment Tracking • CI/CD • Deployment • Monitoring • Drift Feature Stores • Model
Registry • Observability • Reliability • Cost • Security

www.learnmlacademy.com

Free Resource - Share with your network

How to get the most from this handbook

This PDF is designed for serious interview preparation. Each topic combines an interview-ready answer with deeper explanation, examples, diagrams, code, calculations or system trade-offs wherever they improve understanding.

Recommended approach: answer the question yourself first, then compare with the explanation and work through the example.

For coding and SQL questions, state assumptions, approach, complexity and edge cases before writing code. For ML/AI system design, discuss requirements, trade-offs, failure modes, monitoring and security instead of claiming one architecture is always best.

Section	Topics
1. Research basis and further reading	1
TOTAL COVERAGE	1

1. MLOps lifecycle and release discipline

Before debugging scenarios, be clear about what is versioned and how a candidate moves from experiment to production.

Production interview mindset Contain impact first, preserve evidence, compare versions/timelines, identify the first broken contract, verify the fix, then add prevention. 'Retrain the model' is rarely the correct first action.

INTERVIEW QUESTION 2026 SIGNAL

Google Cloud's MLOps architecture explicitly separates CI, CD and continuous training with registry, orchestration and monitoring.

Explain CI, CD and CT in an ML system.

Interview-ready answer

CI validates code, data contracts and model logic; CD safely promotes pipeline/serving artifacts; CT automates retraining when approved triggers and gates are satisfied.

Detailed explanation

ML delivery has more moving parts than ordinary software: code, data, feature definitions, model artifacts, thresholds and evaluation policy. CI should test transformations, schemas and model sanity. CD deploys an immutable compatible bundle. CT may train a candidate, but production promotion still needs quality, reliability and governance gates. Automatic retraining should not imply automatic deployment.

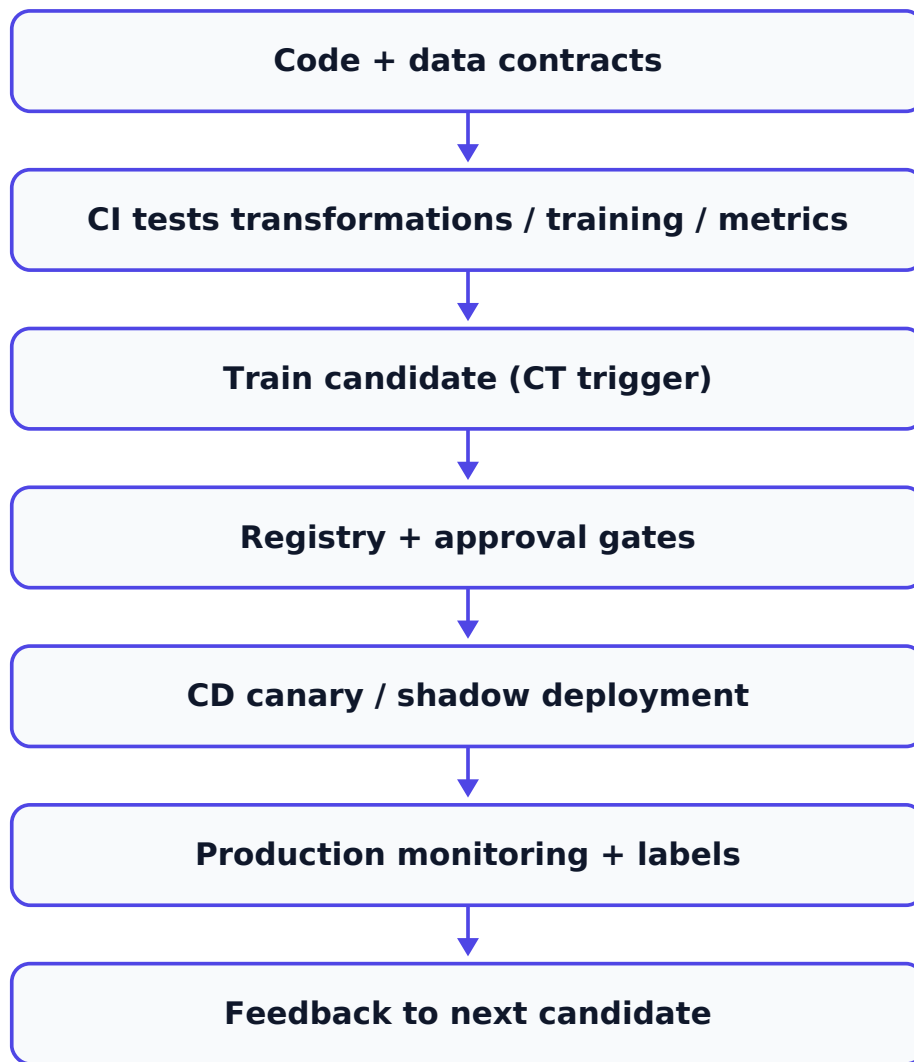


Figure: conceptual flow for this answer.

Example

A daily pipeline detects enough new labeled data, launches training, compares the candidate with champion by slice, registers it, then requires canary evidence before production promotion.

Likely follow-up questions

- What triggers CT?
- What should block promotion even if training succeeded?
- How do you version feature definitions?

Weak answer to avoid

Defining CI/CD/CT only as GitHub Actions, deployment and cron retraining.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

2. Data, feature and evaluation failures

These are the highest-value scenarios because they look like 'model failures' but usually originate in data-time contracts.

PRODUCTION SCENARIO

Offline AUC is excellent, online decisions fail

Stage	What to say
Symptom	Validation AUC is 0.88-0.95 but live business performance is poor.
Likely causes	Target leakage, random split instead of temporal/group split, training-serving feature skew, stale online inputs, threshold mismatch, different user cohort, delayed label definition or intervention feedback.
Evidence to inspect	Replay real production requests using recorded raw inputs; compare online/offline feature vectors, model/threshold versions and label windows; inspect performance by time and cohort.
First safe action	Contain harm by routing to the last known-good model/policy or manual review while preserving failed traces.
Confirmed fix	Repair the confirmed mismatch, rebuild point-in-time evaluation, recalibrate/retune threshold only if evidence supports it, and retrain only when the learned relationship truly needs updating.
Prevention	Point-in-time validation, feature parity tests, shadow/canary release, model/feature/threshold version logging and slice monitoring.

PRODUCTION SCENARIO

Training-serving skew

Stage	What to say
Symptom	The same entity receives materially different predictions offline and online.
Likely causes	Different transformation code, time windows, default values, data sources, feature freshness or normalization statistics.
Evidence to inspect	For one entity and prediction timestamp, compare raw inputs, computed feature values, feature-definition versions and lineage on both paths.
First safe action	Stop promotion or route traffic to the known-good feature/model bundle.
Confirmed fix	Unify versioned feature definitions or repair the service/backfill that violates parity.
Prevention	Shared transformation logic, parity fixtures, feature version/freshness metadata and release-time checks.

PRODUCTION SCENARIO

Stale online feature

Stage	What to say
Symptom	Predictions are technically valid but reflect old customer behavior.
Likely causes	Failed upstream batch, stream lag, stuck consumer, cache TTL bug, source outage or missing freshness contract.
Evidence to inspect	Inspect feature timestamp, source event time, pipeline status, stream lag and stale-rate by feature/source.
First safe action	Use the reviewed stale-data fallback, previous safe feature, or manual/no-action policy; do not silently score with unsafe stale values.
Confirmed fix	Restore ingestion/compute, invalidate stale cache entries and backfill only the affected window.
Prevention	Freshness SLOs, dependency alerts, bounded staleness policy, lineage and runbooks.

PRODUCTION SCENARIO

Schema change causes silent null/default growth

Stage	What to say
Symptom	Inference errors or a sudden rise in defaulted features follows an upstream deployment.
Likely causes	Field rename, type/unit/category change, incompatible enum, nested payload change or consumer silently coercing unknown values.
Evidence to inspect	Compare producer deployment timeline, contract validation failures, payload samples, null/default/range histograms and parser version.
First safe action	Block or quarantine incompatible data instead of accepting silent coercion; route to known-good producer/consumer pair if possible.
Confirmed fix	Deploy compatible schema versions and run a controlled replay/backfill.
Prevention	Versioned contracts, backward/forward compatibility tests, producer-consumer coordination and unknown-category policy.

PRODUCTION SCENARIO

Leakage discovered after deployment

Stage	What to say
Symptom	Historical evaluation is implausibly strong and live quality collapses.
Likely causes	Future feature, target-derived field, duplicate entity across split, label leakage, preprocessing fitted before split or post-outcome events entering features.
Evidence to inspect	Trace feature availability timestamps, dataset construction, split keys, preprocessing fit boundary and ablation results.
First safe action	Stop using the misleading candidate and disclose the invalid evaluation; fall back to a verified model/policy.
Confirmed fix	Rebuild point-in-time data, repeat evaluation from clean boundaries and retrain only from valid features.
Prevention	Leakage checklist, temporal/group splits, feature availability metadata and pipeline-enforced fit/transform boundaries.

3. Serving, LLMOps and retrieval failures

Modern production interviews mix classic latency/capacity questions with LLM-specific memory, retrieval and quality regression.

PRODUCTION SCENARIO

p95 latency spikes while median remains healthy

Stage	What to say
Symptom	Median latency is normal but tail latency breaches the SLO during load.
Likely causes	Queueing, one slow dependency, cold starts, cache misses, oversized inputs, GPU/CPU saturation, garbage collection or retries.
Evidence to inspect	Trace waterfall by percentile, queue depth, dependency p95/p99, payload/token length distribution, cache hit rate and resource saturation.
First safe action	Shed or degrade low-priority work, cap expensive input, protect critical traffic and stop retry amplification.
Confirmed fix	Remove the measured bottleneck: scale capacity, bound input, fix slow dependency, warm/cache safely or redesign batching.
Prevention	Tail-latency alerts, admission control, timeouts, bounded retries, load tests and capacity models.

PRODUCTION SCENARIO2026 SIGNAL

Current KV-cache documentation explicitly treats cache memory as a bottleneck for long-context generation.

GPU OOM only during traffic spikes

Stage	What to say
Symptom	LLM/model workers are stable at normal load but crash when concurrency or context grows.
Likely causes	KV-cache growth, oversized batches, long prompts/outputs, fragmentation, model replica mismatch or scheduler admitting too much work.
Evidence to inspect	GPU memory breakdown, active sequences, prompt/output token distributions, batch sizes, KV-cache occupancy, queue depth and OOM timestamps.
First safe action	Apply admission control, reduce concurrency/batch caps or route overflow to a fallback without repeatedly restarting under the same load.
Confirmed fix	Tune scheduler and memory limits; cap context/output; add replicas, quantization or cache offload only after measuring the actual memory budget.
Prevention	Load tests across token distributions, memory headroom, per-request token budgets and overload policy.

PRODUCTION SCENARIO2026 SIGNAL

Embedding/index migrations are a distinctive modern LLMOps failure mode.

Embedding-model upgrade collapses RAG retrieval

Stage	What to say
Symptom	Recall and answer quality drop immediately after an embedding upgrade.
Likely causes	Mixed old/new vectors, query encoder differs from document encoder, normalization changed, dimension mismatch, incomplete re-embedding or ANN parameters no longer fit.
Evidence to inspect	Embedding/index version, dimensions, vector norms, corpus coverage, fixed retrieval benchmark, shadow production queries and index build logs.
First safe action	Route back to the previous complete index and freeze writes that mix embedding spaces.
Confirmed fix	Build a fully versioned new index, re-embed consistently, validate recall/latency, then atomically switch an alias.
Prevention	Immutable embedding/index versions, migration checklist, coverage gate, shadow evaluation and rollback alias.

PRODUCTION SCENARIO2026 SIGNAL

AI production reliability now requires quality observability, not only infrastructure metrics.

Prompt or model upgrade causes silent quality regression

Stage	What to say
Symptom	Uptime and latency are healthy but users report worse answers after a prompt/model change.
Likely causes	Prompt conflict, changed model behavior, decoding defaults, longer context, tool schema change, evaluator blind spot or new refusal behavior.
Evidence to inspect	Prompt/model hashes, replay dataset, task-success metrics, slice scores, traces, token usage and representative human-rated examples.
First safe action	Stop rollout or return traffic to the known-good prompt/model/tool bundle while preserving failed examples.
Confirmed fix	Repair the verified regression and rerun offline + shadow/canary evaluation before re-promotion.
Prevention	Versioned prompts/models/tools, regression evals, quality gates independent of uptime and representative failure slices.

4. Drift, backfills, feedback loops and safe retraining

The senior-level skill is knowing when not to retrain, and proving that a repair addresses the actual failure.

PRODUCTION SCENARIO

Drift alert fires before labels are available

Stage	What to say
Symptom	PSI or another feature/score distribution metric changes, but outcomes will not arrive for weeks.
Likely causes	Seasonality, population shift, source change, marketing campaign, real covariate shift or data defect. Concept drift is not yet proven.
Evidence to inspect	Feature/source/schema versions, distribution change by slice, score distribution, business process changes and eventually joined labels.
First safe action	Investigate and increase observation; repair clear data defects but do not automatically deploy a retrained model solely because a drift metric moved.
Confirmed fix	If delayed labels confirm performance loss, train/evaluate a candidate using trustworthy outcomes and normal release gates.
Prevention	Contextual drift thresholds, delayed-label joins, champion/challenger evaluation and governed retraining triggers.

PRODUCTION SCENARIO2026 SIGNAL

Senior MLE interviews often probe backfills because they expose lineage and leakage discipline.

Historical backfill changes old features and labels unexpectedly

Stage	What to say
Symptom	A feature repair changes training examples and model performance far outside the intended period.
Likely causes	Non-point-in-time join, mutable source table, changed business logic, timezone cutoff shift or backfill using current state for historical rows.
Evidence to inspect	Dataset manifests, source snapshots, as-of timestamps, feature-definition version, label-generation version and row-level diff.
First safe action	Quarantine the new dataset and preserve the previously reproducible snapshot.
Confirmed fix	Rebuild from versioned historical sources with explicit event-time/as-of semantics; version labels separately from features.
Prevention	Immutable snapshots, point-in-time join tests, backfill review, manifests and reproducible dataset IDs.

PRODUCTION SCENARIO

Recommendation feedback loop narrows content diversity

Stage	What to say
Symptom	Engagement initially rises, then feeds become repetitive and creator/topic diversity collapses.
Likely causes	Training only on exposed items, popularity bias, position bias, insufficient exploration or an objective that rewards only short-term clicks.
Evidence to inspect	Exposure distribution, topic/creator concentration, randomized exploration bucket, long-term retention, diversity metrics and complaint rate.
First safe action	Protect users with diversity/frequency controls and maintain exploration while investigating.
Confirmed fix	Correct labels/objectives, add debiasing or exploration, use multi-objective reranking and include long-horizon metrics.
Prevention	Diversity guardrails, randomized logging, long-term online metrics and feedback-loop monitoring.

PRODUCTION SCENARIO2026 SIGNAL

Agent production interviews increasingly test distributed-systems safety around retries and external actions.

Agent performs duplicate side effects after retries

Stage	What to say
Symptom	The final result appears correct, but the audit log shows duplicate emails, purchases, tickets or database writes.
Likely causes	Non-idempotent tool, lost acknowledgement, blind retry policy, workflow replay from stale checkpoint or missing action receipt.
Evidence to inspect	Tool-call IDs, idempotency keys, external receipts, retry logs, checkpoint/version and target-system audit trail.
First safe action	Disable/gate the affected side-effect tool and prevent additional duplicates while preserving evidence.
Confirmed fix	Introduce stable action IDs, idempotent/deduplicated execution, committed receipts and checkpoint-aware retry logic.
Prevention	Side-effect contracts, replay tests, bounded retries and human approval for high-impact actions.

INTERVIEW QUESTION

When should a production ML system retrain?

Interview-ready answer

Retrain when trustworthy evidence shows the current model no longer meets the decision objective and a candidate trained on appropriate newer data improves it under normal gates - not merely because time passed or a drift statistic moved.

Detailed explanation

Common triggers include enough new labels, known regime change, monitored quality decline or scheduled cadence. Retraining is only candidate creation. Promotion still needs data validation, leakage checks, slice metrics, robustness, cost/latency checks and safe rollout.

Example

A churn model sees a new customer segment after a product launch. Wait for enough representative labels, train a candidate, compare uplift at fixed retention-team capacity, then canary. Do not deploy an unlabeled retrain just because feature PSI rose.

Likely follow-up questions

- When is scheduled retraining appropriate?
- What if labels arrive 90 days later?
- How do you avoid catastrophic rollback loops?

Weak answer to avoid

Equating drift detection with automatic redeployment.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

Deep-dive MLOps tutorial: reproducibility, release, monitoring and recovery

This section connects the ML lifecycle to software engineering, observability, safe release practices and modern LLM/RAG operations.

Study method Do not memorize definitions only. For each concept, explain the idea, work the example, name a failure mode, and state one trade-off. That pattern matches how stronger technical interviews are evaluated.

1. MLOps is lifecycle engineering

MLOps combines data, software, infrastructure and model practices so ML systems can be reproduced, tested, deployed, monitored and improved safely. The model artifact is only one component; data pipelines, feature logic, serving, observability and governance usually dominate operational work.

2. Reproducibility: version code, data and model context

To reproduce a model, record code commit, dependency/environment, data snapshot or lineage, feature definitions, hyperparameters, random seeds where meaningful, evaluation results and artifact checksum.

Exact floating-point reproducibility can be difficult across hardware, but lineage should still explain what produced the model.

3. CI, CD and continuous training

CI validates code, tests, schemas and training components on change. CD promotes tested artifacts to environments. Continuous training retrains based on schedule or conditions and should still pass evaluation and approval gates before deployment.

Do not conflate automatic retraining with automatic promotion.

4. Testing ML systems

Unit tests cover code; data tests cover schema, ranges, nulls and distributions; model tests cover minimum metrics, invariants and behavioral slices; integration tests validate pipeline compatibility; online canaries validate real serving.

ML tests need tolerance because data and stochastic training can vary.

5. Feature pipelines and training-serving skew

Training-serving skew occurs when online features differ semantically from training features due to different code, windows, defaults, time cutoffs or freshness. Shared definitions, point-in-time offline computation and parity tests reduce skew.

Feature freshness should be monitored like any other production SLO.

Deep dive continued

6. Model registry and promotion

A registry tracks candidate artifacts, metadata, evaluation, lineage and deployment status. Promotion is a governance decision: a model should meet task-specific quality, robustness, latency and safety gates.

Keep the ability to roll back to a known-good version quickly.

7. Deployment strategies

Blue-green swaps entire environments; canary sends a small percentage to the new version; shadow mirrors traffic without affecting users; A/B tests measure causal product impact.

Choose according to risk and question: shadow tests infrastructure, A/B tests user impact.

8. Batch, online and streaming inference

Batch inference is efficient for large scheduled scoring. Online inference serves request-time predictions with tight latency. Streaming inference reacts to events continuously. Many products use a hybrid, for example precomputed candidates plus online reranking.

9. Monitoring and observability

Monitor infrastructure, data quality, feature freshness, prediction distributions, drift, model performance when labels arrive, calibration, fairness slices where relevant and business outcomes.

Alerts should have owners, thresholds, runbooks and rollback/mitigation actions.

10. Drift and retraining

Drift metrics indicate change but not necessarily harm. Investigate whether inputs shifted, labels changed, features broke or business behavior changed. Retrain when evidence shows the current model is no longer fit and suitable new labels/data exist.

Evaluate retrained candidates against a frozen benchmark and recent slices.

Deep dive continued

11. LLMOps and RAGOps

LLM systems add prompt/model versioning, token cost, latency, safety evaluation, trace-based quality and non-deterministic outputs. RAG adds index/version lineage, retrieval metrics, source freshness, permissions and citation/grounding evaluation.

Treat prompt, retrieval configuration and model as versioned release artifacts.

12. Incident response

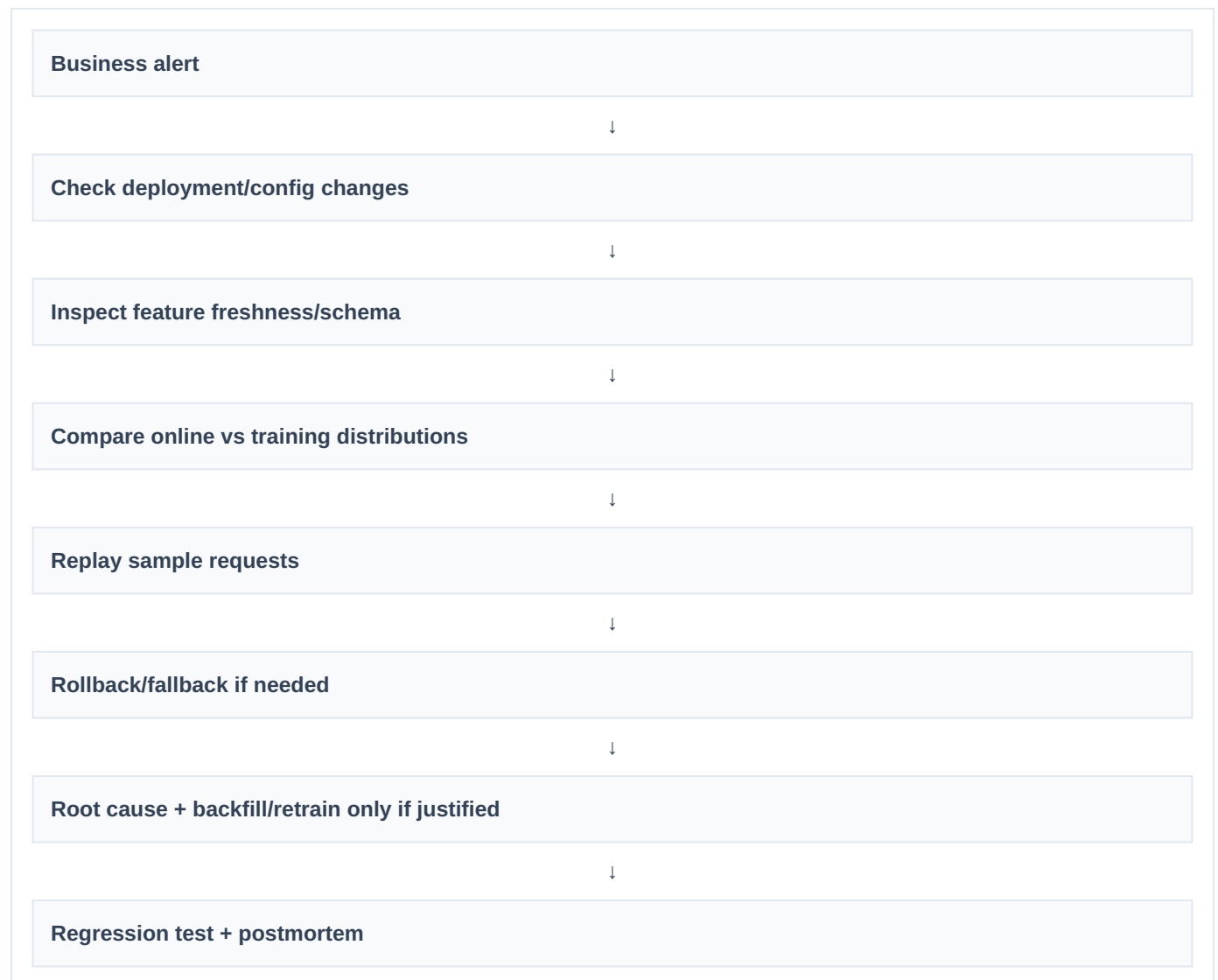
When quality or latency degrades, first stabilize: rollback or fail over if necessary. Then isolate the layer using traces and dashboards. Preserve evidence, identify root cause, patch, add regression tests and document a prevention action.

A strong interview answer distinguishes mitigation from root-cause analysis.

Complete case walkthrough

Case study - production quality incident

Incident: online approval rate drops suddenly although model service health looks green.



A mature answer does not immediately retrain. First check stale/missing features, schema changes, threshold/config changes and upstream data delays. If the model is healthy but the input contract is broken, retraining can make the incident worse.

Additional interview drill

Answer each question aloud in 60-90 seconds. Then add one example, one failure mode and one trade-off.

#	Question	Strong answer should mention
1	CI vs CD vs CT?	test/integrate, deploy, retrain with gates
2	What must be versioned?	code, data/lineage, features, model, config
3	What is training-serving skew?	semantic mismatch between train and online features
4	Shadow vs canary?	no user impact mirror vs limited live impact
5	When retrain?	harmful change + labels/data + candidate passes gates
6	How monitor delayed labels?	input/prediction proxies now, true metrics later
7	What is a model registry?	artifact lineage/evaluation/promotion/deployment metadata
8	How roll back?	immutable versioned artifacts + traffic switch
9	What is data drift?	input distribution change; not automatically concept drift
10	What is RAGOps?	index/retrieval/source/permission/eval lifecycle
11	What should an alert include?	metric, threshold, owner, runbook, severity
12	How handle incident?	mitigate, isolate, root cause, regression test, prevention

Hands-on labs and worked exercises

These exercises turn the concepts into things you can calculate, code, debug or design. Work them before reading the expected reasoning.

Lab 1 - Reproduce a model release

Goal Define the minimum lineage needed to rebuild a model.

Steps

1. Record git commit and environment lock.
2. Record data snapshot/lineage and feature version.
3. Record hyperparameters/seed.
4. Store metrics and artifact checksum.
5. Link deployment to immutable model version.

Expected reasoning / result

Reproducibility is a system property, not just setting `random_seed=42`.

Interview connection

Mention code, data and model/config together.

Lab 2 - Build a release gate

Goal Prevent a “better” model from auto-deploying unsafely.

Steps

1. Require data tests.
2. Require minimum offline quality and slice checks.
3. Benchmark latency/resource use.
4. Run policy/safety tests.
5. Require approval or automated policy decision before promotion.

Expected reasoning / result

Continuous training can be automatic while promotion remains gated.

Interview connection

Separate training automation from release automation.

Lab 3 - Shadow vs canary

Goal Match rollout strategy to risk.

Steps

1. Mirror traffic to new service in shadow and compare outputs/latency.
2. Then route 1-5% live traffic in canary.
3. Monitor rollback criteria.
4. Only then expand or run product experiment.

Expected reasoning / result

Shadow cannot measure user behavior because its outputs do not affect users; canary can expose limited live impact.

Interview connection

Explain what each rollout validates.

Lab 4 - Drift investigation

GoalAvoid retraining reflexively.

Steps

1. Confirm alert is not a pipeline/schema bug.
2. Identify which features shifted.
3. Check prediction distribution.
4. When labels mature, measure slice performance.
5. Retrain only if degradation is real and new data is suitable.

Expected reasoning / result

A drift score indicates difference, not automatically harm.

Interview connection

Strong operators investigate before triggering expensive retraining.

Lab 5 - RAG release regression suite

Goal Version and test retrieval + prompts + model together.

Steps

1. Freeze a labeled query set.
2. Measure Recall@k/reranking.
3. Measure groundedness/citation accuracy.
4. Track latency and token cost.
5. Run prompt-injection and ACL tests.

Expected reasoning / result

A model upgrade can regress RAG even if standalone model benchmarks improve.

Interview connection

LLMOps needs end-to-end application evals.

Last-minute revision sheet

Use this page in the final 30-60 minutes before an interview. Say each answer aloud in one or two sentences; if you cannot, revisit the relevant chapter.

Topic	One-line recall
Incident first step	Contain impact and preserve evidence; do not destroy traces by blindly restarting/retraining.
Skew	Compare identical entity/timestamp across offline and online features.
Staleness	Feature timestamp is part of the contract, not optional metadata.
Latency	Tail traces + queue depth + dependency percentiles reveal p95 problems.
LLM quality	Version prompt, model, tools and eval set; uptime can be green while quality is red.
Drift	A distribution change is a signal to investigate, not proof that model mapping changed.
Backfill	Use immutable snapshots and point-in-time joins.
Agent side effects	Idempotency keys + receipts + bounded retries.

Research basis and further reading

This handbook is original LearnMLAcademy material. Public interview reports and official technical documentation were used to identify current topic emphasis and production practices. The explanations, examples, diagrams and code in this handbook are independently written.

Google Cloud - MLOps Continuous Delivery and Automation Pipelines

<https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>

CI/CD/CT, data and model validation, registry, orchestration, serving and monitoring in production ML systems.

Hugging Face Transformers - KV Cache Strategies

https://huggingface.co/docs/transformers/kv_cache

Current cache types, trade-offs, offloading, quantized cache and iterative generation.

OpenAI - Evaluate Agent Workflows

<https://developers.openai.com/api/docs/guides/agent-evals>

Trace grading, datasets, graders and repeatable agent evaluation workflows.

Aced / Exponent - System Design Interview Prep (2026)

<https://www.tryexponent.com/blog/system-design-interview-guide>

Current system-design prep highlights cost, failure modes, operational judgment, AI/LLM infrastructure and agentic systems.

Google Machine Learning Crash Course

Official ML learning material emphasizes practical exercises, production systems, testing and monitoring in addition to model theory.

Made With ML MLOps Course

Production-oriented curriculum covering data, modeling, testing, reproducibility, serving, CI/CD and monitoring.

Current 2026 interview-preparation resources

Public guides increasingly emphasize production trade-offs, system design, debugging and role-specific reasoning rather than definition-only question banks.

Keep learning at learnmlacademy.com

167 in-depth lessons • Python code examples • Visual diagrams • Projects • Interview preparation • Generative AI • LLM & RAG • Agentic AI • Always free